

---

# Architecture Engineering Document

For: Texas Instruments

**Project: TI TVM Abstraction Layer**



CS Senior Design: Spring 2026

SMU Computer Science Senior Design Team

By:

- Serena DiMartino
- Ria Mukherji
- Zareenah Murad
- Denise Rodriguez

Project Sponsor: Chris Tsongas

# Table of Contents Page

|                                                   |           |
|---------------------------------------------------|-----------|
| <b><i>Company Overview</i></b>                    | <b>3</b>  |
| <b><i>Introduction of Project</i></b>             | <b>3</b>  |
| Key Terminology                                   | 4         |
| The Hardware                                      | 4         |
| <b><i>Requirements &amp; Key Features</i></b>     | <b>5</b>  |
| Functional Requirements                           | 5         |
| Non-functional Requirements                       | 6         |
| <b><i>Architecture &amp; Technical Design</i></b> | <b>7</b>  |
| Introduction                                      | 7         |
| Context Diagram                                   | 8         |
| Architecture Design                               | 9         |
| Compute Routing Explained                         | 10        |
| Technology Stack                                  | 10        |
| <b><i>Implementation</i></b>                      | <b>11</b> |
| Software & Environment Setup                      | 11        |
| How to Build the SDK                              | 11        |
| Hardware Setup                                    | 12        |
| How to Build & Run                                | 12        |
| Coding                                            | 12        |
| Implementation Strategy                           | 12        |
| Implementation Progress                           | 13        |
| System Flow                                       | 14        |
| Testing                                           | 14        |
| <b><i>Handover</i></b>                            | <b>15</b> |
| <b><i>Future Phases</i></b>                       | <b>17</b> |

## Company Overview

Texas Instruments (TI) is a global technology manufacturer that designs and builds semiconductors and integrated circuits. They are headquartered in Dallas, Texas, and have products that span across a range of industries, including industrial, automotive, communications, and consumer electronics. Originally founded in 1930 as Geophysical Service Incorporated, the company's initial focus on seismic technology expanded into broader electronics development, leading to its rebranding as Texas Instruments. TI has played a pivotal role in technological history, introducing innovations such as the first commercial silicon transistor, the invention of the integrated circuit, and the development of early hand-held calculators that helped shape modern computing. Over the decades, TI has continued to refine its semiconductor design and manufacturing capabilities, emphasizing reliability, efficiency, and long-term technology investment.

## Introduction of Project

This project delivers a TVM Abstraction Layer for Texas Instruments' AM69A processor that is designed to simplify and standardize how AI/ML models are deployed to the embedded edge hardware. Currently, there is a major gap between models trained in environments like PyTorch or TensorFlow and the specialized hardware inside the AM69A chip. Our solution creates a unified Python API that allows developers to deploy quantized ONNX models directly onto the AM69A's architecture of ARM Cortex-A72 CPU cores, C7x DSP cores, and Matrix Multiply Accelerator (MMA). Instead of requiring engineers to manually manage low-level hardware configuration, memory routing, and DSP interfaces, this abstraction layer will handle the hardware complexity, so developers can deploy their models more easily.

The TVM solution will easily integrate into a TI developer's existing workflow. An engineer building an edge AI application would be able to train a model in Python, export it to ONNX format, and then use the TVM abstraction layer to compile and run it on the AM69A board. The system automatically routes workloads to the appropriate compute cores while remaining compatible with TI's existing Processor SDK and standard Make-based builds.

For the business, this project creates value by reducing the time required for AI deployment. In the current state, deploying models to the AM69A is highly manual and requires strong technical knowledge. Teams must understand TI's SDK, DSP toolchains, and hardware architecture, which often shifts focus away from model improvement and product innovation. This results in longer development cycles, increased engineering costs, and a complex workflow for engineers. By introducing a hardware-abstracted API through TVM, the future state enables developers to deploy models without writing DSP-specific code, allowing them to leverage the compute resources and move from prototype to production much faster.

## Key Terminology

### The Hardware

- **The Board:** The physical development board that holds the AM69A chip along with memory, ports, and other components needed to run a full embedded Linux system.
- **AM69A:** The main processor chip on the board. It contains all the compute cores below inside a single piece of silicon.
- **ARM Cortex-A72:** The general-purpose CPU cores *inside* the AM69A. Think of it like the brain of a regular computer—it can handle anything but isn't specialized for AI math.
- **C7x DSP:** A specialized math processor *inside* the AM69A, built to handle the heavy repetitive calculations AI models need. Much faster than the ARM cores for those specific tasks.

*Note:* DSP is a Digital Signal Processor that refers to the mathematical manipulation of digitized signals (audio, video, temperature) to improve, modify, or analyze them using algorithms.

- **MMA (Matrix Multiply Accelerator):** Also, *inside* the AM69A, built exclusively for matrix math at the heart of deep learning. The fastest option but only works for specific operations.
- **TDA4 Device:** A family of related TI chips that share the same core architecture as the AM69A. The project codebase should work across these chips with minor adjustments. (the AM69A is a *type* of TDA4 Device)

**Hierarchy: Board → AM69A Chip → ARM Cortex-A72, C7x DSP, MMA**

### The Software Framework

- **ONNX:** A universal file format for AI models, like a ".pdf" for neural networks. It lets you train a model in any framework (PyTorch, TensorFlow, etc.) and export it in a way other tools can read.
- **TVM:** An open-source compiler that takes an ONNX model and translates it into code that can run efficiently on specific hardware. For this project, the hardware is the AM69A's ARM, DSP, and MMA cores.
- **TIDL:** TI's own deep learning library that helps TVM talk directly to the C7x DSP and MMA hardware. It's a low-level bridge between software and specialized chips.
- **SDK (Processor SDK):** TI's full software package for developing on the AM69A. It includes the operating system, drivers, build tools, and documentation. Basically, everything you need to get the board running.

### The Integration Layer (What This Project Builds)

- **Python Bindings:** A wrapper that lets Python code call the underlying C++ TVM runtime. This is what makes it possible to load and run an AI model with just a few lines of Python, without touching any low-level code.
- **.SO File (Shared Object)** — A compiled C++ library file that gets loaded at runtime. This project compiles the TVM integration layer into a .so so Python can import and use it.

## Requirements & Key Features

This project delivers an open-source TVM abstraction layer for the Texas Instruments AM69A processor that enables efficient AI/ML model deployment on embedded hardware. The scope includes system architecture design, TVM integration and runtime development, hardware interface implementation, developer-facing language bindings, build system integration with the TI Processor SDK, and validation through quantized ONNX model execution.

The solution must abstract low-level hardware complexity (C7x DSP cores, MMA accelerator, ARM Cortex-A72 cores) while maintaining performance and portability suitable for embedded environments.

### Functional Requirements

#### Feature 1: Core TVM Integration Layer

|                       |                                                                                                         |
|-----------------------|---------------------------------------------------------------------------------------------------------|
| <b>User Story 1.1</b> | As a developer, I want a clear system architecture, so I understand how models run on the AM69A.        |
| <b>User Story 1.2</b> | As a developer, I want a unified IR interface so I can deploy models without hardware-specific changes. |
| <b>User Story 1.3</b> | As a developer, I want a lightweight runtime, so models run efficiently on embedded hardware.           |

#### Feature 2: TVM Integration & Runtime Implementation

|                       |                                                                                                      |
|-----------------------|------------------------------------------------------------------------------------------------------|
| <b>User Story 2.1</b> | As a developer, I want TVM integrated with the AM69A so models can compile and run on the processor. |
| <b>User Story 2.2</b> | As a developer, I want connection interfaces to ARM and C7x DSP cores so workloads execute properly. |
| <b>User Story 2.3</b> | As a developer, I want support for MMA acceleration (if available) to improve performance.           |

#### Feature 3: Language Bindings & Developer Interface

|                       |                                                                                        |
|-----------------------|----------------------------------------------------------------------------------------|
| <b>User Story 3.1</b> | As a Python developer, I want Python bindings so I can deploy and run models easily.   |
| <b>User Story 3.2</b> | As a C++ developer, I want optional C++ bindings for performance-focused applications. |
| <b>User Story 3.3</b> | As a user, I want a simple API so I do not need low-level hardware knowledge.          |

#### Feature 4: Build System

|                       |                                                                                     |
|-----------------------|-------------------------------------------------------------------------------------|
| <b>User Story 4.1</b> | As a developer, I want the project to build using the TI Processor SDK Make system. |
|-----------------------|-------------------------------------------------------------------------------------|

|                       |                                                                                     |
|-----------------------|-------------------------------------------------------------------------------------|
| <b>User Story 4.2</b> | As a developer, I want the project to build using the TI Processor SDK Make system. |
| <b>User Story 4.3</b> | As a developer, I want the project to build using the TI Processor SDK Make system. |

## Feature 5: Testing & Validation

|                       |                                                                                  |
|-----------------------|----------------------------------------------------------------------------------|
| <b>User Story 5.1</b> | As a tester, I want inference validated on ARM Cortex-A72 cores.                 |
| <b>User Story 5.2</b> | As a tester, I want inference validated on C7x DSP cores and MMA (if supported). |
| <b>User Story 5.3</b> | As a tester, I want at least one quantized ONNX model to run successfully.       |

## Non-functional Requirements

### Performance

- The system should execute inference on a quantized ONNX model within acceptable embedded latency constraints for the AM69A processor.
- The abstraction layer should introduce minimal runtime overhead (target:  $\leq 10\%$  additional overhead compared to direct execution).

### Scalability

- The architecture should support deployment of multiple AI/ML models without modification to the core abstraction layer.
- The design should allow extensions to TI processors with minimal architectural changes.
- The system should support modular expansion for additional accelerators or compute targets.

### Availability & Reliability

- The runtime should execute at least 100 consecutive inference runs without failure.
- The system should provide structured error handling and logging for hardware execution issues.
- Failures in one compute target (e.g., DSP) should not crash the entire runtime.

### Security

- No credentials or sensitive configuration values should be hardcoded.
- All configuration should be managed via external .env files.
- The system should follow secure C/C++ coding practices to prevent unsafe memory operations.

### Usability & Accessibility

- Developers should be able to build and deploy a model using documented instructions without additional training.
- Documentation should include clear build steps, API references, and example deployment scripts.
- The Python API should abstract hardware complexity from end users.

### **Maintainability**

- The codebase should follow a modular structure, separating IR layer, runtime, hardware interfaces, and bindings.
- All public APIs should be documented in Markdown.
- The system should build without modification using the TI Processor SDK toolchain.

### **Data Integrity & Accuracy**

- Inference outputs should remain within acceptable quantization tolerance compared to expected results.
- Model parameters should not be modified during deployment or execution.
- Functional validation should confirm correct inference behavior on supported hardware targets.

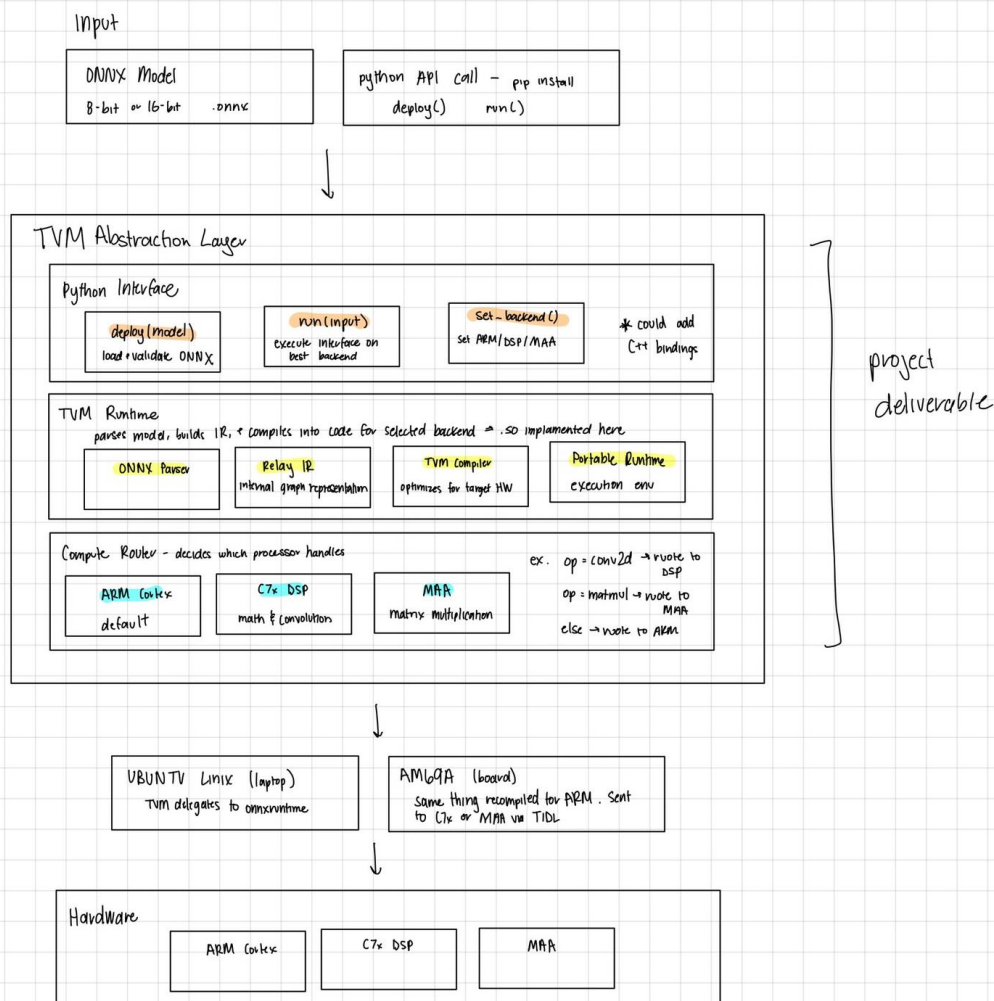
## **Architecture & Technical Design**

### **Introduction**

For our technical solution, we are designing an abstraction layer lets a user deploy an ONNX model to different hardware targets through a simple Python API. The abstraction layer is split into three parts. It starts with a Python Interface that loads the model and picks the right backend, a TVM Runtime that compiles the model into optimized code for that backend, and a Compute Router that decides which processor handles each operation. The end goal was to make it so the user can run their models on the optimal hardware without having to understand or tailor their code accordingly.

Some of our initial challenges have included learning the terminology and understanding how all the components work together. The concepts are interconnected, so it took time to build a clear mental model of how each piece fits into the overall system. We have also been working through an understanding of how the different compilers work and how to send the code to the correct chip.

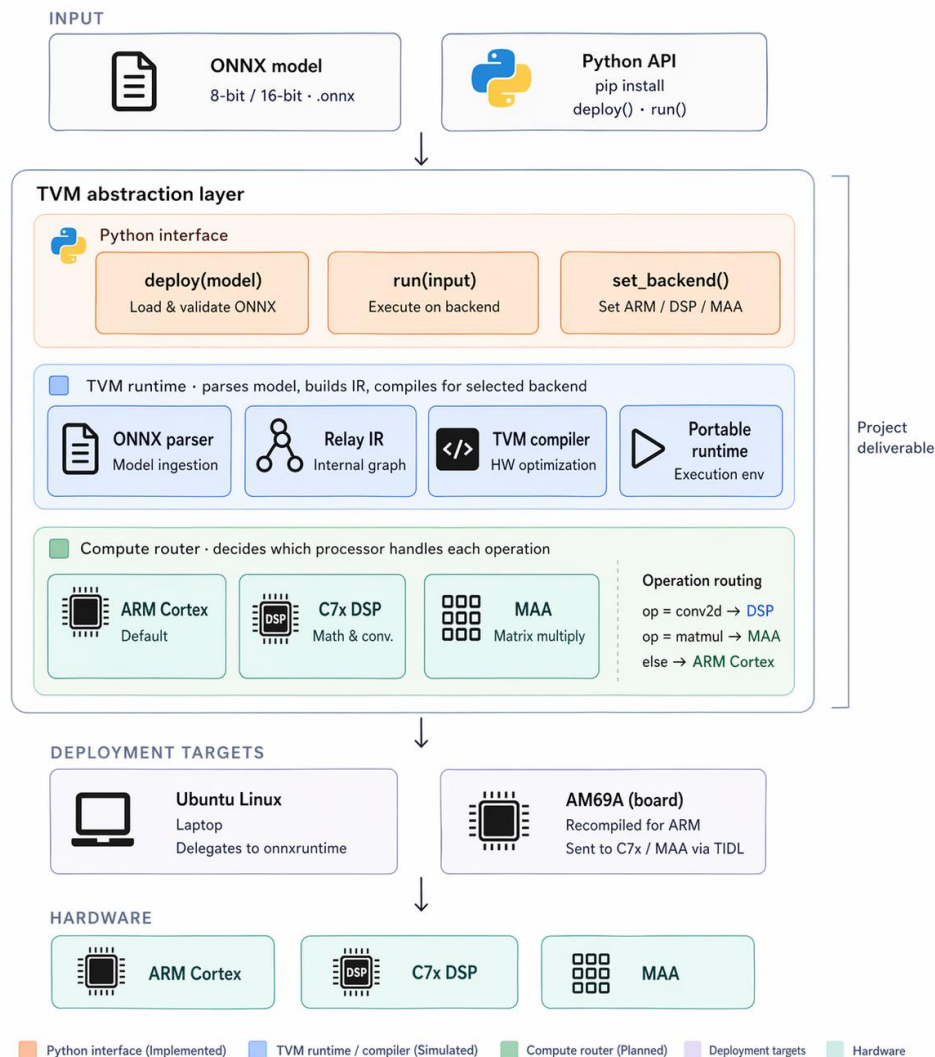
## Context Diagram



This context diagram is one of our later drafts and provides a high-level view of how the system interacts with external components and environments. The system takes a trained ONNX model and user input through a Python interface, then processes that model through the TVM abstraction layer before executing it on the target hardware. The abstraction layer serves as the bridge between developer-facing tools and the underlying AM69A architecture, hiding the complexity of hardware-specific execution. The system interfaces with external frameworks such as ONNX Runtime (for local validation) and the TI Processor SDK (for deployment on embedded hardware). This separation ensures that developers can work within familiar machine learning workflows while the system manages compilation, optimization, and execution across heterogeneous compute resources.



## Architecture Design



### Input Layer

The system begins with two entry points: an ONNX model (either 8-bit or 16-bit) and a Python-based API interface. The ONNX model represents the trained AI/ML model to be deployed, while the Python API provides simple user-facing functions such as `deploy()` and `run()`. This allows users to easily load, validate, and execute models without needing to interact directly with lower-level infrastructure.

### TVM Abstraction Layer (Project Deliverable)

The core deliverable of this project is the TVM abstraction layer, which simplifies deploying optimized AI models onto heterogeneous hardware.

At the top of this layer is the Python Interface, which acts as the main user interaction point. It includes functions like `deploy(model)` to load and validate ONNX models, `run(input)` to execute inference, and `set_backend()` to specify the target hardware (ARM, DSP, or MMA). This

abstracts away the complexity of hardware-specific execution and provides a clean, developer-friendly API. There is also flexibility to extend this layer with C++ bindings for lower-level control.

Beneath this is the TVM Runtime, where most of the system's core processing happens. The ONNX model is first parsed, then transformed into an intermediate representation (Relay IR). From there, the TVM compiler optimizes and compiles the model specifically for the selected hardware backend. Finally, the portable runtime executes the compiled model in a consistent execution environment. This layer ensures that models are both hardware-aware and performance-optimized.

The Compute Router is another key component of the abstraction layer. It determines which hardware unit should execute each operation. For example, convolution operations may be routed to the C7x DSP, matrix multiplications to the MMA accelerator, and all other operations default to the ARM Cortex cores. This routing logic enables efficient utilization of heterogeneous hardware and improves overall performance without requiring manual intervention from the user.

### **Output Layer**

At the final stage, the compiled model is deployed and executed either on a local Ubuntu system (using ONNX Runtime) or on the AM69A board. On the board, the model is executed across ARM cores, DSP, and MMA accelerators based on the routing decisions. This ensures that the model runs efficiently on real hardware while maintaining portability between development and deployment environments.

### [Compute Routing Explained](#)

Compute routing is responsible for determining which hardware component executes each part of a model during inference. The AM69A processor includes multiple compute targets—ARM Cortex-A72 cores, C7x DSP cores, and the MMA accelerator—each optimized for different types of operations. The routing logic analyzes the computational characteristics of each operation in the model and assigns it to the most efficient processor. For example, general-purpose or control-heavy operations are executed on the ARM cores, while highly parallel mathematical operations such as convolutions are routed to the C7x DSP. Matrix-intensive operations, when supported, are offloaded to the MMA accelerator for maximum performance. This approach enables efficient utilization of heterogeneous hardware without requiring the user to manually manage execution, ensuring both performance optimization and ease of use.

### [Technology Stack](#)

The system is built using a combination of technologies that support model portability, hardware abstraction, and efficient execution. Python is used for the primary user-facing API, providing an accessible interface for deploying and running models. C++ is used in conjunction with pybind11 to create a lightweight bridge between Python and the backend execution layer, enabling performance-critical operations and future extensibility. ONNX is used as the

standardized model format, allowing compatibility across different machine learning frameworks, while ONNX Runtime is used in the current prototype to simulate execution. TVM is the intended compiler framework for translating models into optimized code for target hardware. The system is designed to integrate with the TI Processor SDK, which provides the necessary toolchains and libraries for deploying to the AM69A platform. This stack was selected to balance ease of use, modularity, and alignment with future hardware-focused development.

## Implementation

### Software & Environment Setup

To support development on the TI AM69A platform, we established a Linux-based build environment using the TI Processor SDK RTOS (version 11\_00\_00\_06). This SDK provides the necessary toolchains, libraries, and low-level components required to target the C7x DSP cores and MMA (Matrix Multiply Accelerator) units.

The setup process began with downloading and extracting the RTOS SDK, and then configuring the TI C7000 compiler toolchain (cl7x). A custom environment script (ti\_env.sh) was created to streamline this process by exporting required paths (e.g., CGT7X\_ROOT) and updating the system PATH, allowing straightforward access to the compiler and associated tools.

We focused specifically on building the mmalib module within the SDK. This library contains optimized kernels for convolution, linear algebra, and DSP operations targeting the C7x MMA hardware. We successfully compiled the MMALIB components for the C7120 target, generating static library outputs (.lib files). This verified that the toolchain and build environment were correctly configured and capable of compiling code for the target architecture.

Additionally, we identified the c7x-mma-tidl directory within the SDK. This is a key component for future integration, as it bridges low-level acceleration with higher-level deep learning workflows, and will be relevant for connecting our abstraction layer to model execution frameworks.

The environment is now prepared for integrating higher-level components, including the development of a Python interface layer. This will involve creating bindings (likely via C/C++ shared libraries) that expose hardware-accelerated functionality to Python, enabling compatibility with frameworks such as TVM and ONNX Runtime.

### How to Build the SDK

This section outlines the steps required to build the TI SDK components and verify the development environment. Full end-to-end ONNX execution is part of future work, but the current setup establishes the foundation for hardware-accelerated model execution.

First, navigate to the SDK directory and load the TI compiler environment:

```
cd ~/TI/SDK
source ti_env.sh
```

This script configures environment variables such as CGT7X\_ROOT and updates the system PATH to enable use of the TI C7x compiler (cl7x).

To verify the setup:

```
which cl7x
cl7x -version
```

Navigate to the MMALIB directory within the RTOS SDK:

```
cd ti-processor-sdk-rtos-j784s4-evm-
11_00_00_06/mmalib_11_00_00_08
```

Build the C7x MMA libraries using the following command:

```
make -j8 mmalib mmalib_cn common \
  TARGET_CPU=C7120 \
  TARGET_BUILD=release \
  CHECKPARAMS=1 \
  SOC=J7AEP \
  TEST_ENV=EVM
```

Upon successful completion, the compiled libraries will be located in:

```
out/C7120/release/
```

Expected outputs include: mmalib\_C7120.lib, mmalib\_cn\_C7120.lib, and common\_C7120.lib

These artifacts confirm that the SDK is correctly configured and capable of compiling for the C7x DSP with MMA acceleration.

## Hardware Setup

The AM69A board, SD card vs. NFS boot, flashing the .wic image, connecting to the board

## How to Build & Run

Step-by-step build instructions, how to run a sample ONNX model end-to-end

## Coding

### Implementation Strategy

This project primarily focused on designing the system architecture, as discussed with the customer. The goal was not to fully implement every layer, but to clearly define how the system should work end-to-end. In addition to the design, we built a lightweight working prototype to validate the architecture and demonstrate execution flow.

## Coding Standards & Conventions

- Follow PEP 8 for Python (naming, formatting, readability)
- Use PascalCase for classes (e.g., TVMModule) and snake\_case for functions/variables
- Keep functions modular and single-purpose
- Maintain clear separation between layers (Python API, C++ bridge, backend)
- Keep C++ code minimal and focused on integration using pybind11
- Ensure code runs in a clean environment using virtual environments
- Maintain Github, and provide admin access to Chris

## Technology Stack & Development Environment

- Python for the user-facing API and control flow
  - C++ (pybind11) for bridging Python and backend execution
  - ONNX Runtime for executing models as a pass-through backend
  - ONNX as the model format
  - CMake and scikit-build-core for building the C++ extension
  - NumPy for handling input/output
  - GitHub for version control and hosting
- Repository: <https://github.com/riamcodes/tvm-onnx-passthrough>

## Implementation Progress

The system follows a layered architecture aligned with the diagram (orange, yellow, blue).

### Orange Layer – Python Interface (Implemented)

This is the user-facing API and is fully implemented through the TVMModule class. It handles model loading, input formatting, and execution through the run() method.

Example:

```
from tvm_onnx_passthrough import TVMModule
m = TVMModule("model.onnx")
output = m.run({"x": input_array})
```

### Yellow Layer – Compiler / Runtime (Not implemented, simulated)

This layer includes ONNX parsing, Relay IR, compilation, and runtime execution. It was not implemented in this project. Instead, ONNX Runtime is used to simulate this layer by handling model execution internally. This allows the system to function without building a full compiler.

### Blue Layer – Hardware Routing (Not implemented)

This layer is intended to route computation across different hardware targets such as ARM CPU, C7x DSP, and MMA accelerators. It is part of the future design and was not implemented in this phase.

### **C++ Bridge Layer (Implemented)**

This layer connects the Python API to the backend. It is built using pybind11 and compiled as part of the Python package. It enables communication between Python and the execution engine and supports future extensibility.

### System Flow

User (Python)

- TVMModule (orange layer)
- C++ extension (bridge)
- ONNX Runtime (simulating yellow layer)
- Output

A more detailed technical write-up of the system code and implementation approach is provided here: [CodeMasterDoc](#)

Link:

<https://drive.google.com/file/d/1p2gB4yOSHwdWeTNL6v5MtvjLNAoCvrq0/view?usp=sharing>

## Testing

### **Test Concept**

The testing in this project was focused on validating that the architecture works in practice. Since the main goal was designing the system, we did not build a full testing framework. Instead, we tested whether the implemented prototype correctly demonstrates the intended flow.

### **Testing objectives:**

- Confirm the system can be installed and run
- Verify the Python interface works as expected
- Ensure the full pipeline executes without errors

### **Scope of testing:**

- Tested: installation as a Python package, importing the module, running a model end-to-end (Python → C++ → ONNX Runtime)

- Not tested: compiler/runtime layer (yellow), hardware routing (blue), or deployment on ARM/C7x/MMA

### **Testing levels:**

- Integration/System: validating the full pipeline works
- Acceptance: confirming the prototype meets the requirement of demonstrating the architecture

### **Validation Approach**

- Testing was done through simple functional checks rather than formal test cases:
- Installed the package in a clean environment using pip
- Successfully imported the module (from `tvm_onnx_passthrough` import `TVModule`)
- Loaded and ran a test ONNX model using `run()`
- Verified that the output was correct and consistent across runs
- This confirms that the system works end-to-end and that the architecture is correctly implemented at a prototype level.

### **Defect Management & Reporting**

Most issues came from setup and build errors (e.g., dependencies, headers, CMake). These were fixed during development by adjusting the environment and code until the system ran successfully. GitHub can be used for tracking issues in future phases.

### **Future Testing (Next Steps)**

The next phase should include more structured testing, such as:

- Unit tests for individual components
- More detailed integration tests
- Testing on ARM and hardware environments
- Validation of the compiler/runtime layer once implemented

Testing in this phase was focused on making sure the system is installable and runs correctly end-to-end. More comprehensive testing should be added once additional parts of the system are built in future group's iterations.

## **Handover**

**Handover Plan** – how will Company take ownership of code

The codebase is stored in a GitHub repository and is structured as a pip-installable Python package. This allows the customer or future developers to easily download, install, and run the system in a clean environment.

**Repository:** <https://github.com/riamcodes/tvm-onnx-passthrough>

Access to the repository has been shared, and our manager Chris has been added as an admin to ensure the company maintains ownership and control moving forward.

To take ownership, the next team should:

- Clone the repository
- Create a Python virtual environment
- Run pip install . from the root directory
- Run the demo/example script to verify the system works

All required dependencies and build steps are included in the repository to support a straightforward setup.

**Transition Plan** – how will you transition, what has been done, what will be done

All deliverables have been shared with the customer, including the architecture diagram, supporting documentation, and the working prototype. The system design and key decisions are documented so the next team can understand both what was implemented and what is planned. The code is available in the GitHub repository, and access has been provided, so the customer and future students can continue development without needing additional handoff.

This project focused on designing the system architecture and validating it through a working prototype. The following was completed:

- Defined the full layered architecture (orange, yellow, blue)
- Implemented the Python interface (TVModule)
- Built the C++ bridge using pybind11
- Used ONNX Runtime as a backend to simulate execution
- Created an installable Python package
- Verified end-to-end execution of a sample model

The next team will have access to all documentation, including this report and the additional architecture write-up, which explains how the system is structured and how the layers map to the implementation.

Remaining work includes:

- Implement the compiler/runtime layer (yellow)



- Add hardware routing and support (blue layer: ARM, C7x, MMA)
- Expand testing (unit, integration, performance)
- Improve documentation and examples
- Port the system to target hardware environments

Where to start (for next students)

The next team should begin by running the current system:

1. Clone the repository and install the package using pip
2. Import TVMModule and run the example model
3. Review the architecture diagram and documentation to understand how the layers map to the code

Once the system is understood, development can continue by either extending the backend (replacing ONNX Runtime) or focusing on deployment and hardware integration. The current implementation provides a clear and working foundation to build on.

## Future Phases

The most important next step is completing the code based on the architecture that has already been designed. The customer has already discussed future work around extending the current prototype rather than redesigning the system from scratch. This includes moving beyond the current proof of concept and implementing the remaining architectural layers.

Recommended future phases include:

- Complete the compiler/runtime layer (yellow)
- Add hardware routing and support for ARM, C7x, and MMA (blue layer)
- Port the current package to target hardware environments
- Expand testing beyond the prototype level
- Improve documentation, examples, and installation flow
- A practical next step would be to first build out the remaining code in the existing architecture before adding more advanced optimizations. Since the current prototype already validates the interface and execution flow, the next team can focus on extending the implementation rather than reworking the design.
- The customer has already indicated interest in continuing this work through hardware-focused phases, especially ARM porting and C7x/MMA integration. Because of this, future teams should treat the current system as a foundation and continue development in the same direction.